A.D. 1308
unipg
DIPARTIMENTO
DI INGEGNERIA

Progetto Finale di

Deep Learning and Robot Perception

Corso di Laurea Magistrale in Ingegneria Informatica e Robotica – A.A. 2024-2025

Dipartimento di Ingegneria

docente

Prof. Gabriele Costante

Monocular Depth Estimation

Deep Learning Challenge on an Unreal Engine dataset

studente

Federico Ombelli

1 Problem Description

The goal of this project is to develop a *deep neural network* to estimate depth information from a single RGB image. The dataset provided includes images of a virtual world generated using *Unreal Engine* with their corresponding depth maps, the training set has 3469 samples while the validation set has 600 samples.

In the following report I will first give an overview of the *Monocular Depth Estimation* problem and existing models, then I will describe the proposed model and discuss the obtained results.

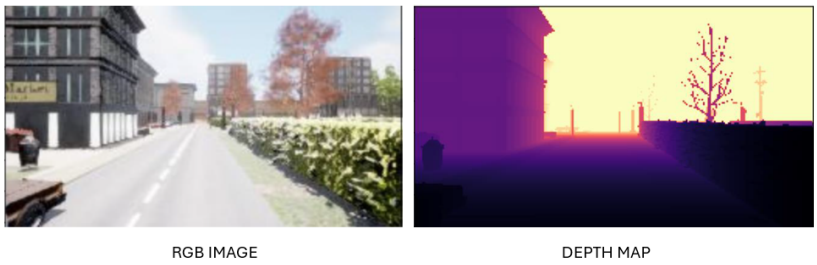

Figure 1.1: Example of a dataset sample

1.1 Monocular Depth Estimation

The process of estimating the distance of objects in a scene from a single camera viewpoint is known in computer vision as **Monocular Depth Estimation**. It is a difficult task because a model has to overcome different challenges: the main one is the mapping ambiguity between the 2D image and the 3D scene to be reconstructed, in addition the parameters of the camera used to take the image influence the 2D representation of the original scene, another challenge is brightness inconsistency between images, two images with the same depth map may have different color and illumination distributions.

Depth Estimation has important applications like 3D reconstruction, autonomous driving, virtual and augmented reality and over the years many different solution to this problem has been proposed but significant improvements have only been achieved with the advent of *deep learning* methods . In fact traditional methods required specialized hardware and sensors like *LiDAR* to capture depth information from a scene, however recent approaches show that, using Deep Learning methods, it is possible to reach good performance and accuracy in estimating depth using only 2D images, reducing cost and making possible to integrate it in small devices such as wearable.

1.2 Review of recent State of the Art Models

All the recent Deep Learning models for Monocular Depth Estimation can be categorized into three main paradigms [2]:

- **Supervised learning**: the model is trained on a labeled dataset containing images and ground truth depth maps and it learns a direct mapping between raw image pixel and depth values, usually this method can achieve high accuracy but requires large labeled datasets;
- **Self-supervised learning**: this method uses structure and information in unlabeled data to estimate depth without requiring ground truth depth maps, multiple views of the same scene are needed so the datasets used usually include image pairs or sequences;
- **Semi-supervised learning**: this method is based on self-supervised learning but uses both labeled data and unlabeled data for training.

There are two main depth estimation categories [1]:

- **Absolute** (or metric) **depth estimation**: that provides exact depth values that represent real-world distances from the camera;
- **Relative depth estimation**: that predicts the relative order of the objects in a scene estimating which is closer or further but without providing precise distance measures.

Regarding the **model architectures**, a common pattern used in recent models is the **Encoder-Decoder**: the encoder acts as a feature extractor that extracts relevant information from the input image, compressing them into a *latent vector* that the decoder uses to generate the output depth map.

Convolutional neural networks are often used as both as encoder and as decoder, the encoder is usually a *pre-trained* convolutional network like *ResNet* to extract robust features for the decoder, following the transfer-learning technique. An example of a model with this architecture is the first version of *MiDaS* [3].

A common architecture is also **U-Net**, which consists in a convolutional network based on the encoder-decoder pattern with added connections between layers of the two sections, called *skip connections*, that enable the decoder to see also features at lower levels of the encoder. An example of this is the *DenseDepth* model [6].

Many recent models instead adopt **transformer-based** architectures, like the *Vision Transformers*, that use the attention mechanism and the global receptive field typical of this architecture to achieve even better performance.

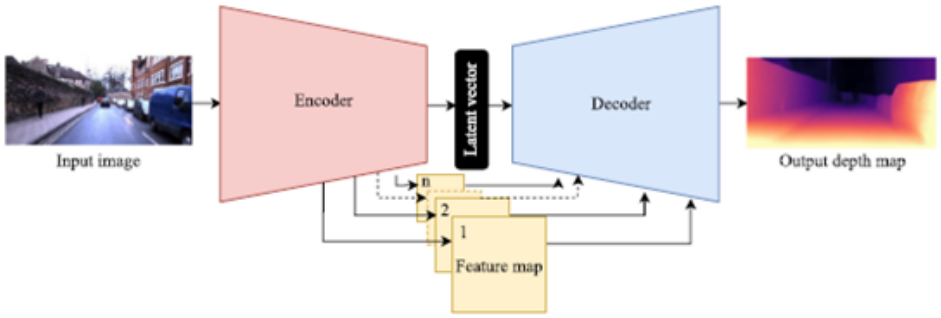

Figure 1.2: Representation of an Encoder-Decoder architecture using skip connections between the two sections

Finally, several models use a **hybrid architecture**, for example combining *Vision Transformer* and *CNN*, in order to leverage the strengths of each architecture. For example the last versions of *MiDas 3.1* uses different transformer encoders combined with a convolutional decoder [5].

Currently the State-of-the-Art models that achieve the best performance are:

- **ZoeDepth** for absolute depth estimation, based on the last version of *MiDaS 3.1* (a relative depth estimation model) adding on top of that a module to estimate metric depth [7];
- **Depth Anything v2** for relative depth estimation, it is based on a DPT architecture (*Dense Prediction Transformer*) [4] with the DINOv2 vision transformer as encoder, implements some innovative concepts such as the *teacher-student* paradigm and it is trained on a large dataset of labeled synthetic images and unlabeled real images [8, 1].

2 The proposed model

The proposed model to solve the task follows the commonly used **Encoder - Decoder** pattern and in particular uses a **U-Net** style architecture based on the *DenseDepth* model proposed by *Alhashim and Wonka* [6], with some changes. While *DenseDepth* uses *DenseNet-169* as the encoder backbone, for this model I chose a more traditional **ResNet-50** as backbone, popular for depth estimation tasks. The main reason for this choice is the limited resources available for training, in fact, although the *DenseNet* architecture provides better performances, it requires more GPU memory due to its complex architecture and dense connectivity between layers.

The other design choices made for the model are described in the following sections.

2.1 Encoder-Backbone

As already mentioned, the encoder part of the model is based on a **ResNet-50** network pretrained on *ImageNet*, from which the last 2 layers (*average pooling* and *fully connected*) were removed because they are only used for classification [10].

The full architecture of the original ResNet-50 is summarized in the image 2.1.

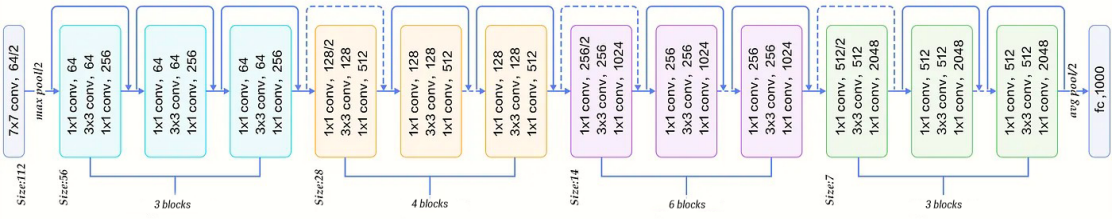

Figure 2.1: Full architecture of ResNet-50

The encoder maintains the first 5 sections of ResNet, from *layer0* to *layer4*, so the encoder outputs 2048 feature maps at 1/32 of the input resolution.

To take advantage of the features learned by ResNet during the training on the huge *ImageNet* dataset and to reduce the risk of overfitting, due to the small dataset available, I chose to initialize the encoder with pretrained weights from PyTorch (*ResNet50_Weights.IMAGENET1K_V2*) and to freeze it for the first epochs, following the **transfer-learning** technique.

In addition, to adapt the training to the specific synthetic dataset, I unlocked the last two encoder layers (*layer3* and *layer4*) after the first 5 epochs once the training was more stable. This is a common practice in Deep Learning because it makes possible to **fine-tune** the last layers of the network that encode more dataset specific features, while retaining the pre-trained weights on the earlier layers that encodes more low-level features like edges or corners [11].

For the fine-tuning of the encoder I set a lower learning rate than the one used to train the decoder, as described in the section 2.3.

2.2 Decoder

The decoder receives in input the compressed representation of the image generated by the encoder and reconstructs the full resolution depth map using a sequence of

upsampling and convolutional layers.

In particular, the decoder consists of 5 main **convolutional blocks**, each preceded by an **upsampling layer** that progressively upsamples the feature maps to the output resolution.

- Each **Convolutional Block** is made by a sequence of *Convolution - Batch-Normalization - ReLu* repeated twice, the parameters used for the convolutions are: *kernel 3x3*, *stride 1* and *padding 1*, as used in ResNet.
- The **first 3 upsampling layers** are made by a *Transposed Convolution* with *kernel 4x4*, *stride 2* and *padding 1* followed by *BatchNormalization* and *ReLU*, each layer double the dimensions of the feature maps and split in half the number of channels.
- the **last 2 upsampling layers** instead are simple *bilinear upsampling layers* as also used in *DenseDepth*, that simply double the dimensions of the feature maps. This reduces the "check board artifacts" sometimes generated by *ConvTranspose2d* layers on the output depth maps.

As mentioned above the decoder follows a **U-Net** design, implementing direct connections between encoder and decoder layers, called *skip connections*, that enable the decoder to see also features at lower levels that can be lost during the downsampling phases. This connection are implemented taking the outputs from encoder layers 1 through 3, reducing their channel dimensionality with a *1x1 convolution* and finally concatenating them with the corresponding decoder layers.

The following scheme summaries how the decoder layers work:

- **encoder_layer4** (out: 1/32 dim, 2048 ch) → **Upsample ConvTranspose** (1/16 dim) → **concatenate with encoder_layer3** (reduced to 512 ch) → **decoder_layer1** (in: 1536 ch - out: 512ch)
- **decoder_layer1** (out: 512 ch) → **Upsample ConvTranspose** (1/8 dim) → **concatenate with encoder_layer2** (reduced to 256 ch) → **decoder_layer2** (in: 512 ch - out: 256 ch)
- **decoder_layer2** (out: 256 ch) → **Upsample ConvTranspose** (1/4 dim)→ **concatenate with encoder_layer1** (reduced to 128 ch) → **decoder_layer3** (in: 256 ch - out: 128 ch)
- **decoder_layer3** (out: 128 ch) → **Bilinear upsample** (1/2 dim) → **decoder_layer4** (in: 128 ch - out: 64 ch)
- **decoder_layer4** (out: 64 ch) → **Bilinear upsample** (1/1 dim) → **decoder_layer5** (in: 64 ch - out: 32 ch)
- **decoder_layer5** (out: 32 ch) → **out_conv** (out: 1 ch)

2.3 Training strategy

To train the model, first of all I applied to the training and evaluation sets the preprocessing used in ResNet pre-training, like resizing the images and depth maps to 224 x 224 and applying normalization only to RGB images, this is useful because the first encoder layers are frozen and normalizing the input images in the same way as during pre-training can improve performance.

In addition, given the small size of the dataset (3469 training samples), I applied to the training set a simple **data augmentation** technique like the random horizontal flipping, to reduce the risk of overfitting.

The **Loss function** used for training is based on the one proposedin the *DenseDepth* paper[6], with some modifications like a Berhu loss term in place of the L1 term. The final loss is defined as the weighted sum of three terms (where $\hat{y}$ is the estimated depth map and y is the *ground truth*):

$$L(y, \hat{y}) = \alpha L_{BERHU}(y, \hat{y}) + \beta L_{GRAD}(y, \hat{y}) + \gamma L_{SSIM}(y, \hat{y})$$

- $L_{GRAD}(y, \hat{y})$ is a L1 loss defined over the gradients of the depth map
- $L_{SSIM}(y, \hat{y})$ is based on the **SSIM** (Structure Similarity Index Measure) a common used metric for image reconstruction tasks that measure the similarity between two images, it is mainly used to evaluate performance but in many cases it is also used as a loss term. In this case, as the SSIM has a range of 0-1, the loss term is defined as follows:

  $$L_{SSIM}(y, \hat{y}) = \frac{1}{2}(1 - SSIM(y, \hat{y}))$$

- $L_{BERHU}(y, \hat{y})$ is the **BerHu loss** (or Reverse Huber) that combines the L1 and L2 loss using the L1 when the error is between a specified range and L2 otherwise. It is used in many depth estimation works because it gives better results than L1 or L2 loss alone [2, 9]. This loss is defined as the mean over all pixels of the following terms:

  $$L_{BERHU}(e_i) = \begin{cases} |e_i|, & \text{if } |e_i| \leq c \\ \dfrac{e_i^2 + c^2}{2c}, & \text{if } |e_i| > c \end{cases}$$

  where $e_i = \hat{y}_i - y_i$ is the error for pixel i and $c = \delta \cdot \max_i |e_i|, \quad \delta = 0.2$.

The weights assigned to each term are defined empirically based on the tests as $\alpha = 0.2; \beta = 0.5; \gamma = 0.5$.

The other parameters used for training are:

- **Adam** optimizer with differentiated values for encoder and decoder
  - learning rate *1e-4* and weight decay *1e-3* for the decoder
  - lower learning rate **1e-6** and weight decay **1e-5** for the fine-tuning of encoder layers
- **Cosine annealing** scheduler to dynamically decrease the learning rate over the epochs, following a cosine curve from the values defined above to a minimum value of 1e-7;
- **batch size** = 8;
- last 2 encoder layers (*layer3* and *layer4*) unlocked after the first 5 epochs.

3 Results

The performance of the proposed model are evaluated using two commonly used metrics for depth estimation:

- **RMSE** (Root Mean Squared Error), where lower values indicates better predictions;
- **SSIM** (Structure Similarity Index Measure), a complex similarity measure also included as a term in the training loss as described in section 2.3, it ranges from 0 to 1 where values closer to 1 indicates higher similarity between the predicted and the ground truth depth maps.

After a training session of 40 epochs, the model achieved the best results at **epoch 34**. The **quantitative results**, computed on the validation set, are the following:

- $RMSE_{VAL} = 2.76$
- $SSIM_{VAL} = 0.57$

These results indicates that the proposed model is able to reproduce good depth estimations even though it does not reach the performance of *DenseDepth* or of recent state-of-the-art-models. This was expected considering the limited size of the dataset used for training and the relatively simple architecture compared to recent models.

However, it is possible to identify potential improvements in the current model, such as:

- applying more advanced data augmentation strategies to increase the generalization ability of the model even with a small dataset;
- using more powerful backbones for the encoder, such as a *Vision Transformer*, that demonstrated better performance in recent works;
- refining the loss function, to improve the estimation for very close or very far objects, that the current model struggles to predict.

4 Bibliography

[1] huggingface.co - *Metric and Relative Monocular Depth Estimation: An Overview* - `https://huggingface.co/learn/computer-vision-course/en/unit8/monocular_depth_estimation`

[2] U. Rajapaksha, F. Sohel, H. Laga, D. Diepeveen, M. Bennamoun, 2024 - *Deep Learning-based Depth Estimation Methods from Monocular Image and Videos: A Comprehensive Survey* - `https://dl.acm.org/doi/10.1145/3677327`

[3] R. Ranftl, K. Lasinger, D. Hafner, K. Schindler, C. Koltun, 2020 - *Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-shot Cross-dataset Transfer* - `https://arxiv.org/pdf/1907.01341`

[4] R. Ranftl, A. Bochkovskiy, V. Koltun, 2021 - *Vision Transformers for Dense Prediction* - `https://arxiv.org/pdf/2103.13413`

[5] R. Birkl, D. Wofk, M. Müller, 2023 - *MiDaS v3.1– A Model Zoo for Robust Monocular Relative Depth Estimation* - `https://arxiv.org/pdf/2307.14460`

[6] I. Alhashim, P. Wonka, 2019 - *High Quality Monocular Depth Estimation via Transfer Learning* - `https://arxiv.org/abs/1812.11941`

[7] S.F. Bhat, R. Birkl, D. Wofk, P. Wonka, M. Müller, 2023 - *ZoeDepth: Zero-shot Transfer by Combining Relative and Metric Depth* - `https://arxiv.org/pdf/2302.12288`

[8] L. Yang, B. Kang, Z. Huang, Z, Zhao, X. Xu, J. Fneg, H. Zhao, 2024 - *Depth Anything V2* - `https://arxiv.org/pdf/2406.09414`

[9] I. Laina, C. Rupprect, 2016 - *Deeper Depth Prediction with Fully Convolutional Residual Networks* - `https://ieeexplore.ieee.org/abstract/document/7785097`

[10] K. He, X. Zhang, S. Ren, J. Sun, 2015 - *Deep Residual Learning for Image Recognition* - https://arxiv.org/pdf/1512.03385

[11] CS231n: Deep Learning for Computer Vision - *Transfer Learning and Fine Tuning* - https://cs231n.github.io/transfer-learning/